Databases Course

Homework 3 Report

Variant: Assistantships

Author: Pasichnyk Nazar
Date: 2026-04-13

Table of Contents

1. Introduction
2. Step 1: Specify In-Scope Stakeholders, Tasks, Input and Output Documents
3. Step 2: Elaborate External Views for In-Scope Stakeholders
4. Step 3: Harmonize External Views in a Refined Conceptual Model
5. Step 4: Refine Database Schema
6. Step 5: Normalize Database to 3NF
7. Step 6: Populate Normalized Database
8. Step 7: Develop DBS Application: 3+ Forms for Data Manipulation
9. Step 8: Develop DBS Application: 3+ Reports for Output Documents
10. Step 9: Develop Tests and Test the DBS Application
11. Summary
12. Conclusion
13. References
14. Annex

1. Introduction

This report documents the full HW3 workflow for the Assistantships variant: scoping, external views, harmonized conceptual model, refined schema, normalization proofs, database population, application forms and reports, and end-to-end testing.

The work is aligned to HW3 guidelines and grading penalties, with explicit Inputs/Outputs per step and reproducible evidence files in the repository.

Repository: https://github.com/dark-neonus/db_hw_3_pasichnyk

2. Step 1: Specify In-Scope Stakeholders, Tasks, Input and Output Documents

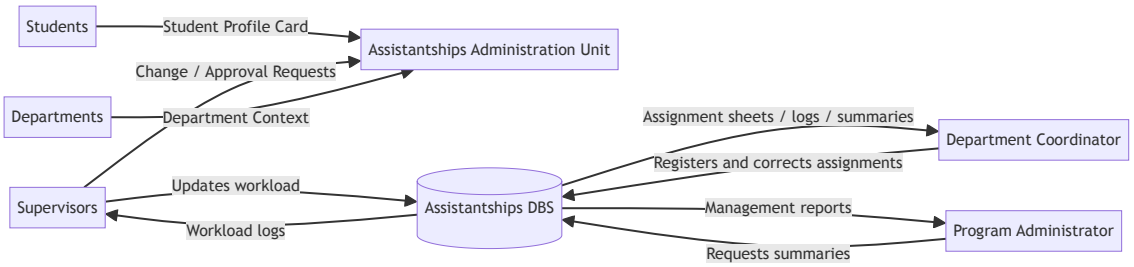
Inputs

- HW1 Assistantships conceptual model and requirements.
- HW2 relational schema and operational SQL artifacts.

Outputs

- In-scope stakeholders and scoping diagram.
- Automated tasks with stakeholder, input document, output document, and automation degree.
- Structured input documents (3+).
- Structured output documents (3+).

2.1 Scoping Diagram and In-Scope Units



2.2 In-Scope Stakeholders and Roles

1. Department Coordinator: registers assistantship assignments and handles correction/termination requests.
2. Supervisor: updates workload data and validates change rationale.
3. Program Administrator: generates allocation and contract-level summaries.

2.3 In-Scope Tasks Selected for Automation

Task ID	Task	Stakeholder	Input Document(s)	Output Document(s)	Automation Degree
T1	Register new assistantship assignment	Department Coordinator	Assistantship Registration Form, Student Profile Card	Assistantship Assignment Sheet	Full
T2	Update assistantship workload	Supervisor	Assistantship Change Request	Workload Change Log	Full
T3	Delete invalid/terminated assignment	Department Coordinator	Assistantship Termination Request	Deletion Confirmation Record	Full
T4	Generate managerial summaries	Program Administrator	Report Filter Request	Allocation Summary, Contracts Summary, Workload Audit	Full

2.4 Input Documents and Structures

1. Assistantship Registration Form: assistantship_id, registration_number, semester, academic_year, hours_per_week, assistantship_type, student_id, supervisor_id, department_id.
2. Student Profile Card: student_id, first_name, last_name, email, enrollment_date, program.
3. Assistantship Change Request: assistantship_id, new_hours, changed_by, reason.
4. Assistantship Termination Request: student_id, assistantship_id, initiated_by, reason.
5. Report Filter Request: semester, academic_year, department_id, assistantship_type.

2.5 Output Documents and Structures

1. Assistantship Assignment Sheet: assistantship_id, student_name, supervisor_name, department_name, semester, hours_per_week, assistantship_type.
2. Workload Change Log: change_id, assistantship_id, old_hours, new_hours, changed_by, change_date, reason.
3. Deletion Confirmation Record: deleted_student_id, deleted_assistantship_ids, timestamp, operator.
4. Assistantship Allocation Summary: department_name, assistantship_type, total.
5. Current Contracts Summary: student_name, assistantship_id, hours_per_week, semester.

2.6 Why These Tasks Were Selected

These tasks are the complete in-scope information flow where structured input documents are transformed into either:

- persistent database updates (T1-T3), or
- generated output documents/reports (T4).

This matches the HW3 requirement that selected operations are information-processing operations automated by the DBS.

3. Step 2: Elaborate External Views for In-Scope Stakeholders

Inputs

- Stakeholders, tasks, and documents from Step 1.
- Baseline conceptual model from HW1/HW2.

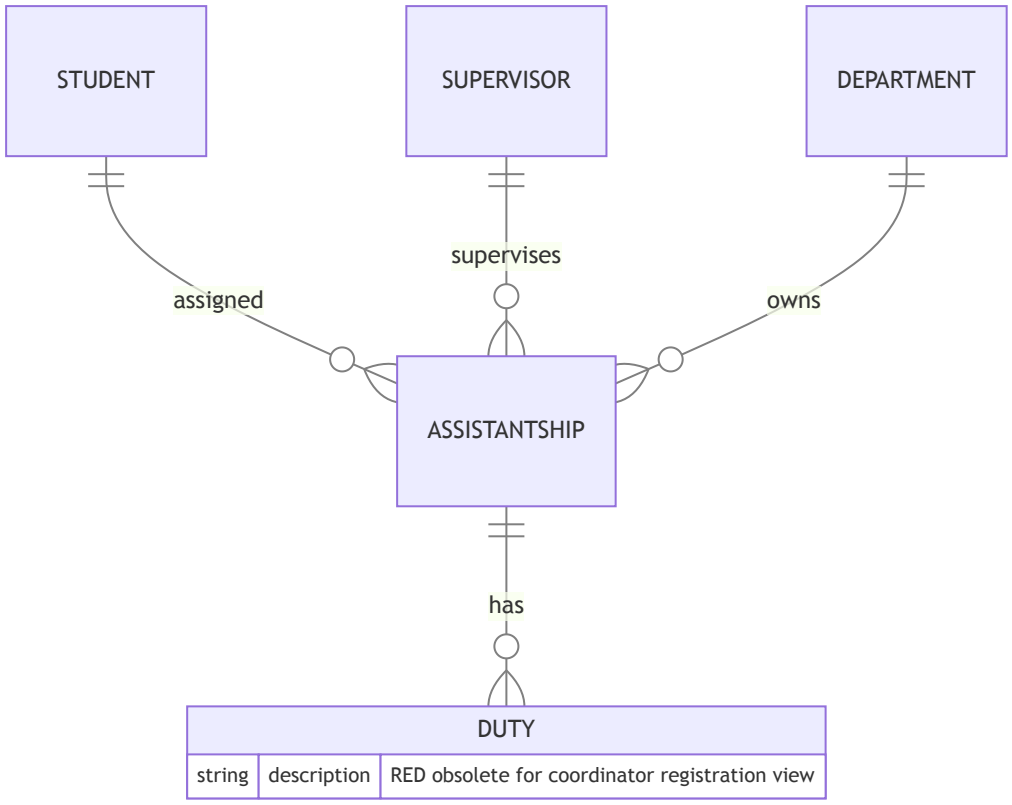
Outputs

- External view per in-scope stakeholder.
- Input model fragments with obsolete elements marked RED.
- External view elements/attributes added for stakeholder needs marked GREEN.

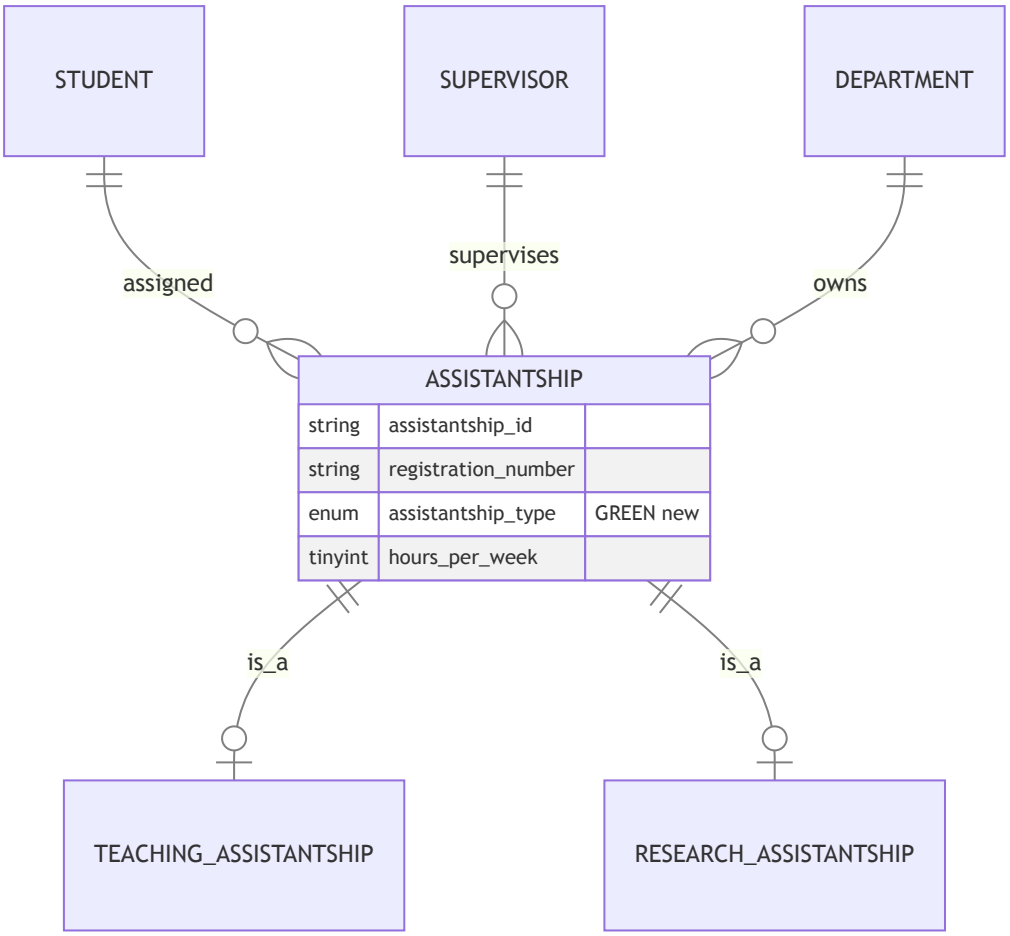
Note: in markdown diagrams, RED/GREEN are shown by labels due text-only rendering.

3.1 External View: Department Coordinator

Input CM Fragment (with obsolete-for-view elements)



External View (new elements in GREEN)

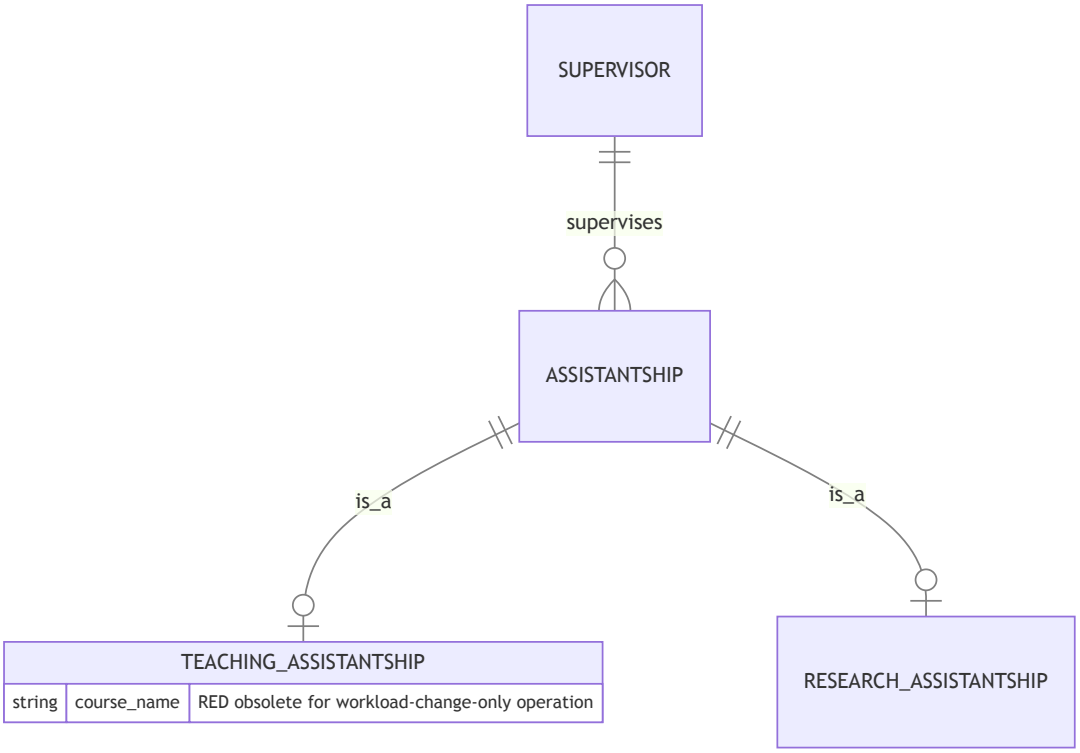


Data Objects Alignment

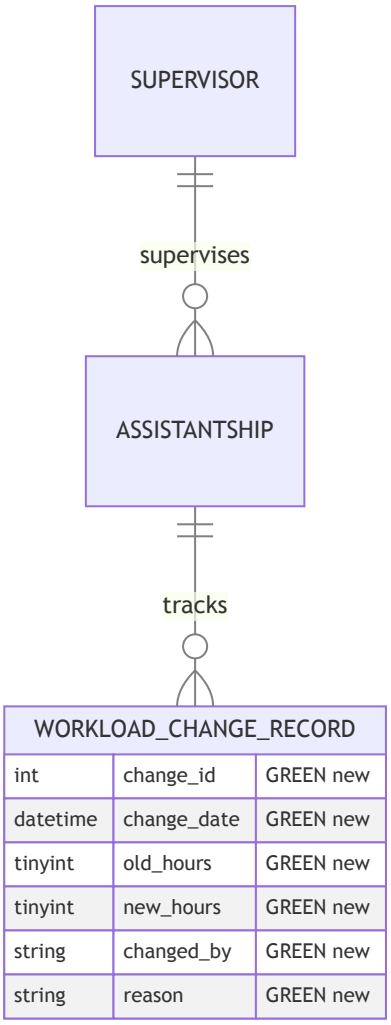
- Input docs used: Assistantship Registration Form, Student Profile Card, Assistantship Termination Request.
- Output docs produced: Assistantship Assignment Sheet, Deletion Confirmation Record.

3.2 External View: Supervisor

Input CM Fragment (with obsolete-for-view elements)



External View (new elements in GREEN)

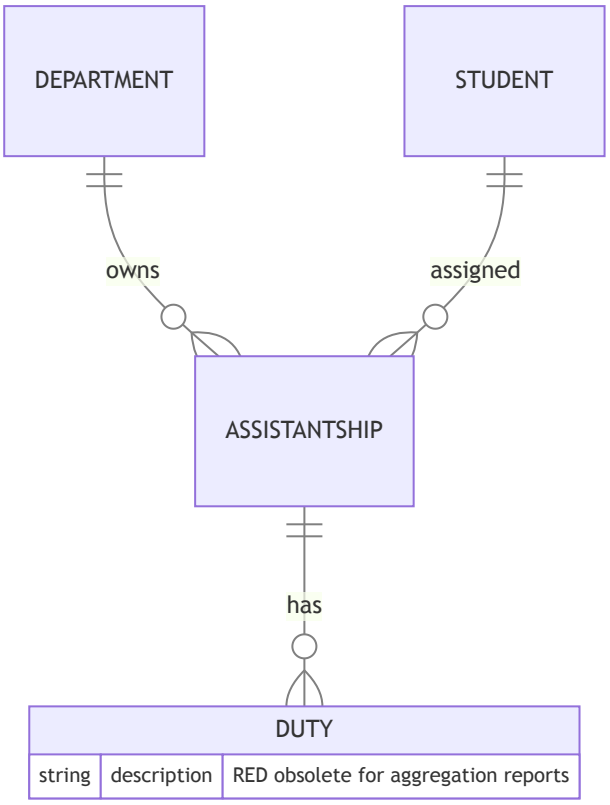


Data Objects Alignment

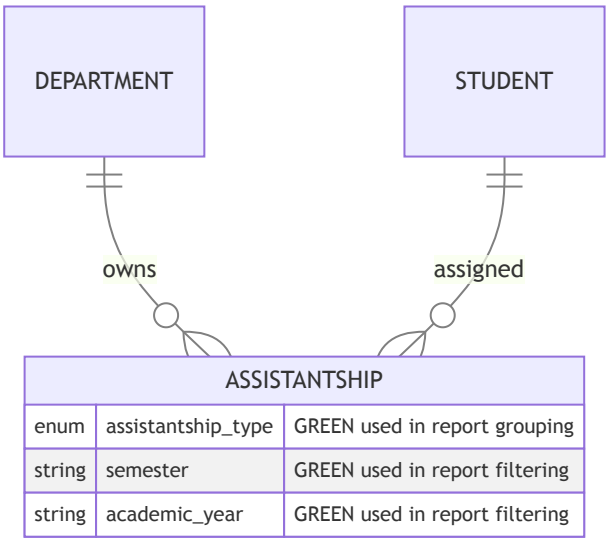
- Input doc used: Assistantship Change Request.
- Output doc produced: Workload Change Log.

3.3 External View: Program Administrator

Input CM Fragment (with obsolete-for-view elements)



External View (new elements in GREEN)



Data Objects Alignment

- Input doc used: Report Filter Request.
- Output docs produced: Allocation Summary, Contracts Summary, Workload Audit.

4. Step 3: Harmonize External Views in a Refined Conceptual Model

Inputs

- External views from Step 2.

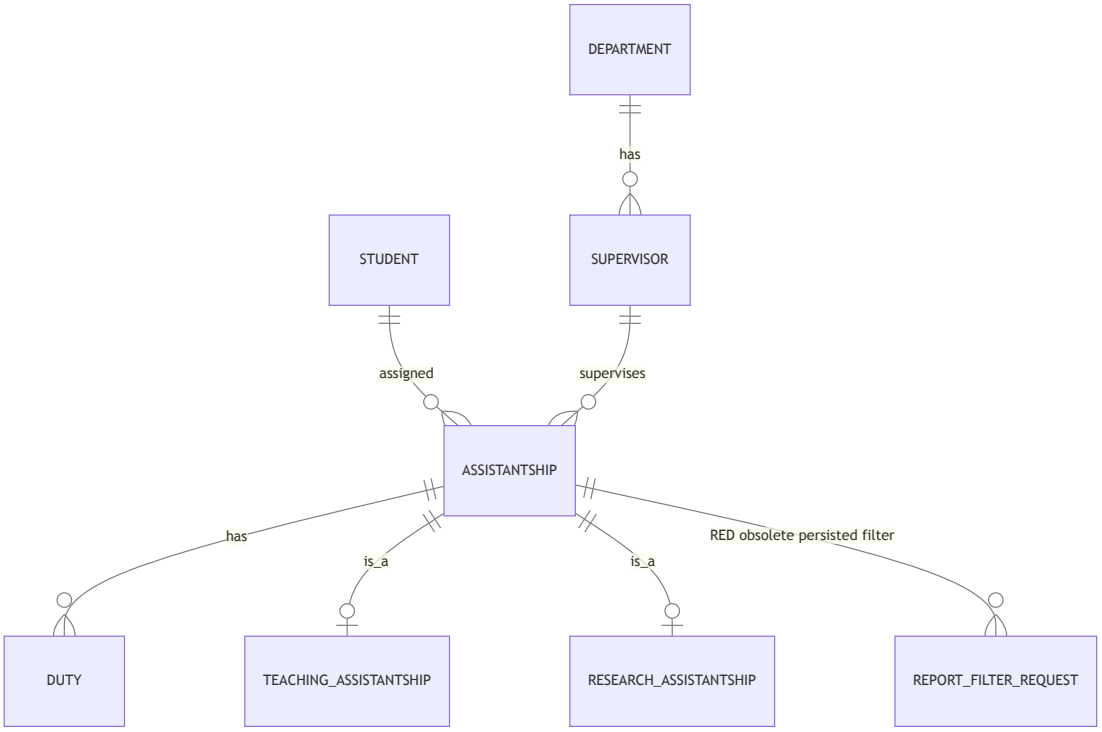
Outputs

- Harmonized conceptual model with resolved conflicts.
- Input vs refined comparison with RED obsolete and GREEN new labels.

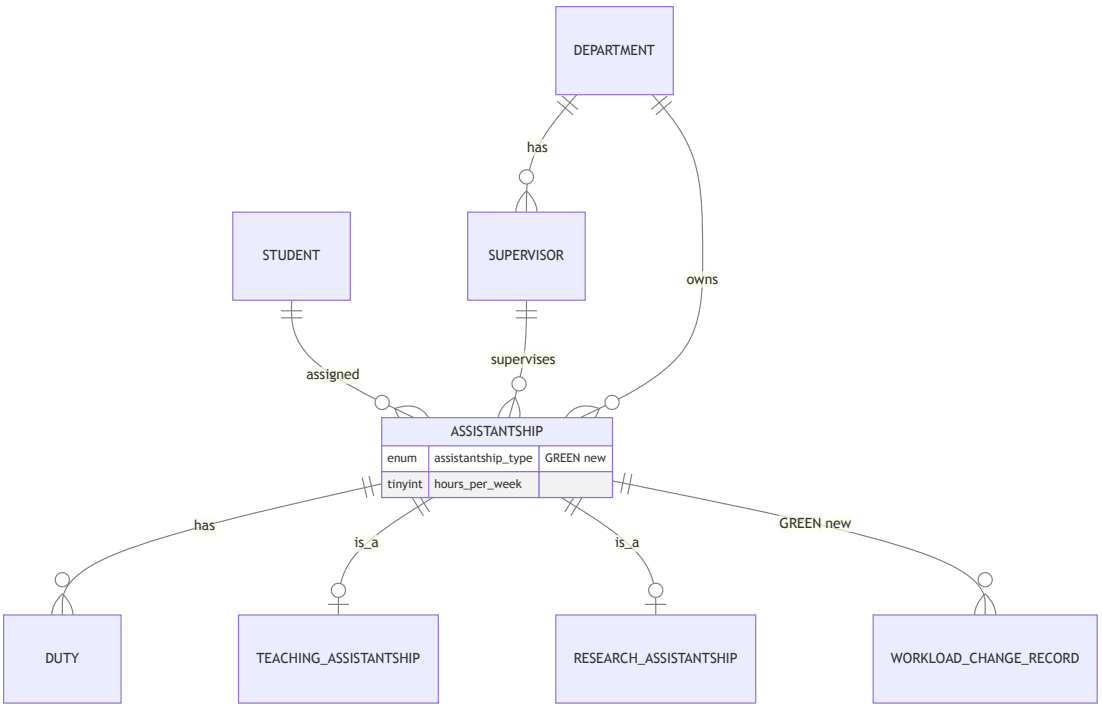
4.1 Harmonization of Conflicts and Overlaps

Conflict/Overlap	Resolution	Rationale
Type representation for TA/RA across coordinator/admin views	Keep single assistantship_type on assistantship + existing TA/RA specialization tables	One shared semantic source for both operations and reports
Workload updates were previously state-only (no history)	Add workload_change_record entity	Required to prove updates and generate audit report
Report filter fields considered for persistence	Keep filters transient (request parameters), not persistent tables	Prevent redundant data and transitive dependencies

4.2 Input Model (Comparison Baseline)



4.3 Refined Conceptual Model



5. Step 4: Refine Database Schema

Inputs

- Refined conceptual model from Step 3.
- Existing HW2 schema baseline.

Outputs

- Refined logical schema.
- DDL for target tables.
- DDL execution proofs.

5.1 Mapping Refined Conceptual Elements to Logical Tables

Conceptual Element	Logical Table	Mapping Note
Department	department	Strong entity
Supervisor	supervisor	FK to department
Student	student	Strong entity
Assistantship + type	assistantship	assistantship_type added
Duty	duty	Weak entity with composite PK
TA specialization	teaching_assistantship	PK=FK to assistantship

RA specialization	research_assistantship	PK=FK to assistantship
Workload change history (new)	workload_change_record	New table with FK to assistantship

5.2 DDL Queries per Target Table

Exact DDL queries (implemented in `sql/step4_refined_schema.sql`) are listed below per target table.

1. `department`

```sql
CREATE TABLE department
(
  department_id   INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  department_name VARCHAR(100) NOT NULL,
  faculty_name    VARCHAR(100) NOT NULL
) ENGINE = InnoDB;
```

2. `supervisor`

```sql
CREATE TABLE supervisor
(
  supervisor_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  first_name    VARCHAR(50) NOT NULL,
  last_name     VARCHAR(50) NOT NULL,
  title         VARCHAR(20) NOT NULL,
  email         VARCHAR(100) NOT NULL UNIQUE,
  department_id INT UNSIGNED NOT NULL,
  CONSTRAINT fk_supervisor_department
    FOREIGN KEY (department_id) REFERENCES department(department_id)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE = InnoDB;
```

3. `student`

```sql
CREATE TABLE student
(
  student_id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  first_name      VARCHAR(50) NOT NULL,
  last_name       VARCHAR(50) NOT NULL,
  email           VARCHAR(100) NOT NULL UNIQUE,
  enrollment_date DATE NOT NULL,
  program         VARCHAR(100) NOT NULL
) ENGINE = InnoDB;
```

4. `assistantship`

```sql
CREATE TABLE assistantship
(
```

```
  assistantship_id    VARCHAR(15) NOT NULL PRIMARY KEY,
  registration_number VARCHAR(20) NOT NULL UNIQUE,
  semester            ENUM('Fall', 'Spring', 'Summer') NOT NULL,
  academic_year       VARCHAR(9) NOT NULL,
  hours_per_week      TINYINT UNSIGNED NOT NULL,
  assistantship_type  ENUM('TA', 'RA') NOT NULL,
  student_id          INT UNSIGNED NOT NULL,
  supervisor_id       INT UNSIGNED NOT NULL,
  department_id       INT UNSIGNED NOT NULL,
  CONSTRAINT fk_assistantship_student
    FOREIGN KEY (student_id) REFERENCES student(student_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_assistantship_supervisor
    FOREIGN KEY (supervisor_id) REFERENCES supervisor(supervisor_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_assistantship_department
    FOREIGN KEY (department_id) REFERENCES department(department_id)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE = InnoDB;
```

5. `duty`

```
CREATE TABLE duty
(
  assistantship_id VARCHAR(15) NOT NULL,
  duty_id          SMALLINT UNSIGNED NOT NULL,
  description      VARCHAR(255) NOT NULL,
  PRIMARY KEY (assistantship_id, duty_id),
  CONSTRAINT fk_duty_assistantship
    FOREIGN KEY (assistantship_id) REFERENCES assistantship(assistantship_id)
    ON UPDATE CASCADE ON DELETE CASCADE
) ENGINE = InnoDB;
```

6. `teaching_assistantship`

```
CREATE TABLE teaching_assistantship
(
  assistantship_id       VARCHAR(15) NOT NULL PRIMARY KEY,
  course_code            VARCHAR(20) NOT NULL,
  course_name            VARCHAR(100) NOT NULL,
  grading_responsibility BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT fk_ta_assistantship
    FOREIGN KEY (assistantship_id) REFERENCES assistantship(assistantship_id)
    ON UPDATE CASCADE ON DELETE CASCADE
) ENGINE = InnoDB;
```

7. `research_assistantship`

```
CREATE TABLE research_assistantship
(
```

```
  assistantship_id VARCHAR(15) NOT NULL PRIMARY KEY,
  project_name     VARCHAR(150) NOT NULL,
  research_topic   VARCHAR(255) NOT NULL,
  grant_funded     BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT fk_ra_assistantship
    FOREIGN KEY (assistantship_id) REFERENCES assistantship(assistantship_id)
    ON UPDATE CASCADE ON DELETE CASCADE
) ENGINE = InnoDB;
```

8. `workload_change_record` (GREEN new table)

```
CREATE TABLE workload_change_record
(
  change_id        INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  assistantship_id VARCHAR(15) NOT NULL,
  change_date      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  old_hours        TINYINT UNSIGNED NOT NULL,
  new_hours        TINYINT UNSIGNED NOT NULL,
  changed_by       VARCHAR(50) NOT NULL,
  reason           VARCHAR(255) NOT NULL,
  CONSTRAINT fk_workload_assistantship
    FOREIGN KEY (assistantship_id) REFERENCES assistantship(assistantship_id)
    ON UPDATE CASCADE ON DELETE CASCADE
) ENGINE = InnoDB;
```

5.3 Proof of DDL Execution (per target table)

Execution command:

```
docker exec -i assistantships-db-hw-3-mysql mysql -uroot -psecret <
sql/step4_refined_schema.sql
```

Execution artifacts used for proof:

- `evidence/step4/show_tables.txt`
- `evidence/step4/describe_department.txt`
- `evidence/step4/describe_supervisor.txt`
- `evidence/step4/describe_student.txt`
- `evidence/step4/describe_assistantship.txt`
- `evidence/step4/describe_duty.txt`
- `evidence/step4/describe_teaching_assistantship.txt`
- `evidence/step4/describe_research_assistantship.txt`
- `evidence/step4/describe_workload_change_record.txt`

Per-table execution screenshots:

`department`:

Step 4 DDL Execution Proof

Table: **department**
Status: table created and visible in SHOW TABLES output.

DESCRIBE department

```
department_id   int unsigned    NO      PRI     NULL    auto_increment
department_name varchar(100)    NO              NULL
faculty_name    varchar(100)    NO              NULL
```

`supervisor`:

Step 4 DDL Execution Proof

Table: **supervisor**
Status: table created and visible in SHOW TABLES output.

DESCRIBE supervisor

```
supervisor_id   int unsigned    NO      PRI     NULL    auto_increment
first_name      varchar(50)     NO              NULL
last_name       varchar(50)     NO              NULL
title   varchar(20)     NO              NULL
email   varchar(100)    NO      UNI     NULL
department_id   int unsigned    NO      MUL     NULL
```

`student`:

Step 4 DDL Execution Proof

Table: **student**
Status: table created and visible in SHOW TABLES output.

DESCRIBE student

```
student_id      int unsigned    NO      PRI     NULL    auto_increment
first_name      varchar(50)     NO              NULL
last_name       varchar(50)     NO              NULL
email   varchar(100)    NO      UNI     NULL
enrollment_date date    NO              NULL
program varchar(100)    NO              NULL
```

`assistantship`:

Step 4 DDL Execution Proof

Table: **assistantship**
Status: table created and visible in SHOW TABLES output.

DESCRIBE assistantship

```
assistantship_id        varchar(15)     NO      PRI     NULL
registration_number     varchar(20)     NO      UNI     NULL
semester        enum('Fall','Spring','Summer')  NO              NULL
academic_year   varchar(9)      NO              NULL
hours_per_week  tinyint unsigned        NO              NULL
assistantship_type      enum('TA','RA') NO              NULL
student_id      int unsigned    NO      MUL     NULL
supervisor_id   int unsigned    NO      MUL     NULL
department_id   int unsigned    NO      MUL     NULL
```

`duty`:

Step 4 DDL Execution Proof

Table: **duty**
Status: table created and visible in SHOW TABLES output.

DESCRIBE duty

```
assistantship_id        varchar(15)     NO      PRI     NULL
duty_id smallint unsigned       NO      PRI     NULL
description     varchar(255)    NO              NULL
```

`teaching_assistantship`:

Step 4 DDL Execution Proof

Table: **teaching_assistantship**
Status: table created and visible in SHOW TABLES output.

DESCRIBE teaching_assistantship

```
assistantship_id        varchar(15)     NO      PRI     NULL
course_code     varchar(20)     NO              NULL
course_name     varchar(100)    NO              NULL
grading_responsibility  tinyint(1)      NO              0
```

`research_assistantship`:

Step 4 DDL Execution Proof

Table: **research_assistantship**
Status: table created and visible in SHOW TABLES output.

DESCRIBE research_assistantship

```
assistantship_id        varchar(15)     NO      PRI     NULL
project_name    varchar(150)    NO              NULL
research_topic  varchar(255)    NO              NULL
grant_funded    tinyint(1)      NO              0
```

`workload_change_record`:

Step 4 DDL Execution Proof

Table: **workload_change_record**
Status: table created and visible in SHOW TABLES output.

DESCRIBE workload_change_record

```
change_id       int unsigned    NO      PRI     NULL    auto_increment
assistantship_id        varchar(15)     NO      MUL     NULL
change_date     datetime        NO              CURRENT_TIMESTAMP       DEFAULT_GENERATED
old_hours       tinyint unsigned        NO              NULL
new_hours       tinyint unsigned        NO              NULL
changed_by      varchar(50)     NO              NULL
reason  varchar(255)    NO              NULL
```

6. Step 5: Normalize Database to 3NF

Inputs

- Refined schema from Step 4.

Outputs

- 1NF proof.
- 2NF proof.
- 3NF proof.

6.1 1NF

Rule: attributes must be atomic.

Typical source fragment (non-1NF candidate):

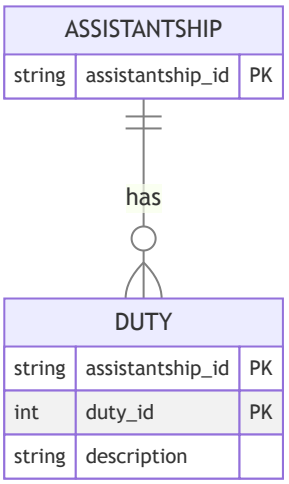
```
assistantship_before_1nf(assistantship_id, ..., duties_list)
```

Conceptual/schema fragment illustration:

ASSISTANTSHIP_BEFORE_1NF			
string	assistantship_id	PK	
string	duties_list		non-atomic list

Resulting 1NF fragment:

```
assistantship(assistantship_id, ...)
duty(assistantship_id, duty_id, description)
```



Proof:

- All current table attributes are scalar/atomic types (INT, VARCHAR, DATE, ENUM, DATETIME, BOOLEAN/TINYINT).
- No repeating groups exist in a single column.

6.2 2NF

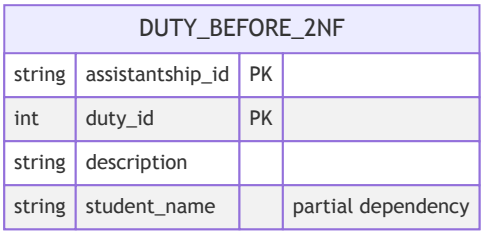
Rule: for composite keys, each non-key attribute must depend on the full key.

Typical source fragment (partial dependency risk):

```
duty_before_2nf(assistantship_id, duty_id, description, student_name)
```

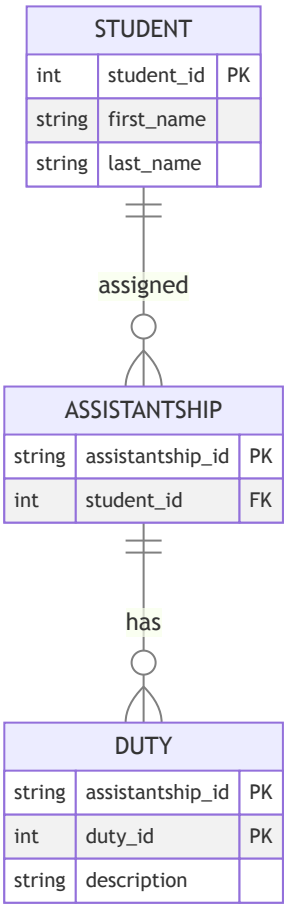
where student_name depends only on assistantship_id, not full composite key.

Conceptual/schema fragment illustration:



Resulting 2NF fragment:

```
duty(assistantship_id, duty_id, description)
student(student_id, first_name, last_name, ...)
assistantship(assistantship_id, student_id, ...)
```



Proof:

- duty has composite PK (assistantship_id, duty_id), and description depends on both.
- Other tables use single-column PKs, so partial dependencies are not possible.

6.3 3NF

Rule: no transitive dependencies from key to non-key through another non-key.

Typical source fragment (transitive dependency risk):

```
supervisor_before_3nf(supervisor_id, department_id, department_name)
```

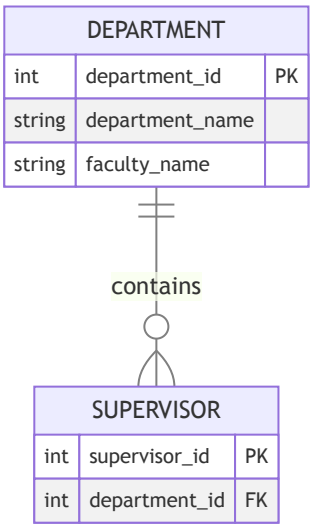
where supervisor_id -> department_id -> department_name.

Conceptual/schema fragment illustration:

SUPERVISOR_BEFORE_3NF			
int	supervisor_id	PK	
int	department_id		
string	department_name		transitive via department_id

Resulting 3NF fragment:

```
supervisor(supervisor_id, ..., department_id)
department(department_id, department_name, faculty_name)
```



Proof:

- department_name and faculty_name are stored only in department.
- supervisor and assistantship store only department_id as FK.

- workload_change_record attributes depend on change_id only.

Conclusion: the refined schema is in 3NF.

7. Step 6: Populate Normalized Database

Inputs

- Normalized schema (Step 5).

Outputs

- Population script.
- Proof of successful execution.
- Data sufficiency rationale for Steps 7-9.

7.1 Source Data and Population Queries

Population queries are provided in `sql/step6_populate_data.sql` using INSERT statements per table source.

Per-table source data blocks and population statements:

1. `department` : INSERT block `-- 1. Department`
2. `supervisor` : INSERT block `-- 2. Supervisor`
3. `student` : INSERT block `-- 3. Student`
4. `assistantship` : INSERT block `-- 4. Assistantship`
5. `duty` : INSERT block `-- 5. Duty`
6. `teaching_assistantship` : INSERT block `-- 6. Teaching_Assistantship`
7. `research_assistantship` : INSERT block `-- 7. Research_Assistantship`
8. `workload_change_record` : INSERT block `-- 8. WorkloadChangeRecord (NEW)`

Execution command:

```
docker exec -i assistantships-db-hw-3-mysql mysql -uroot -psecret <
sql/step6_populate_data.sql
```

7.2 Population Proof and Table Results

Execution artifacts used for proof:

- `evidence/step6/populate_apply.log`
- `evidence/step6/count_department.txt`
- `evidence/step6/count_supervisor.txt`
- `evidence/step6/count_student.txt`
- `evidence/step6/count_assistantship.txt`
- `evidence/step6/count_duty.txt`
- `evidence/step6/count_teaching_assistantship.txt`
- `evidence/step6/count_research_assistantship.txt`
- `evidence/step6/count_workload_change_record.txt`

Row counts after Step 6 load on clean DB:

- department: 10
- supervisor: 12
- student: 12

- assistantship: 12
- duty: 12
- teaching_assistantship: 7
- research_assistantship: 5
- workload_change_record: 3

Per-table population screenshots:

`department` :

Step 6 Population Proof

Table: **department**
Rows loaded:
10

Sample rows (LIMIT 5)

```
1       Computer Science        Faculty of Applied Sciences
2       Data Science    Faculty of Applied Sciences
3       Software Engineering    Faculty of Applied Sciences
4       Mathematics     Faculty of Natural Sciences
5       Physics Faculty of Natural Sciences
```

`supervisor` :

Step 6 Population Proof

Table: **supervisor**
Rows loaded:
12

Sample rows (LIMIT 5)

```
1       Oleksandr       Tkachenko       Prof.   oleksandr.tkachenko@ucu.edu.ua  1
2       Mariya  Kovalchuk       Dr.     mariya.kovalchuk@ucu.edu.ua     1
3       Andriy  Shevchenko      Assoc. Prof.    andriy.shevchenko@ucu.edu.ua    2
4       Nataliya        Bondar  Prof.   nataliya.bondar@ucu.edu.ua      3
5       Serhiy  Melnyk  Dr.     serhiy.melnyk@ucu.edu.ua        4
```

`student`:

Step 6 Population Proof

Table: **student**
Rows loaded:
12

Sample rows (LIMIT 5)

```
1       Ivan    Petrenko        ivan.petrenko@ucu.edu.ua        2023-09-01      Computer Science BSc
2       Anna    Shevchuk        anna.shevchuk@ucu.edu.ua        2022-09-01      Data Science MSc
3       Taras   Boyko   taras.boyko@ucu.edu.ua  2023-09-01      Software Engineering BSc
4       Olha    Koval   olha.koval@ucu.edu.ua   2021-09-01      Mathematics MSc
5       Mykola  Savchuk mykola.savchuk@ucu.edu.ua       2024-09-01      Physics BSc
```

`assistantship`:

Step 6 Population Proof

Table: **assistantship**
Rows loaded:
12

Sample rows (LIMIT 5)

```
AST-2025-0001   REG-25-001-     Fall    2024-2025       10      TA      1       1       1
AST-2025-0002   REG-25-002-     Fall    2024-2025       20      RA      2       3       2
AST-2025-0003   REG-25-003-     Spring  2024-2025       15      TA      3       4       3
AST-2025-0004   REG-25-004-     Spring  2024-2025       10      TA      4       5       4
AST-2025-0005   REG-25-005-     Fall    2024-2025       20      RA      5       6       5
```

`duty`:

Step 6 Population Proof

Table: **duty**
Rows loaded:
12

Sample rows (LIMIT 5)

```
AST-2025-0001   1       Grading assignments
AST-2025-0002   1       Data cleaning and preprocessing
AST-2025-0003   1       Assisting in labs
AST-2025-0004   1       Grading quizzes
AST-2025-0005   1       Setting up lab equipment
```

`teaching_assistantship`:

Step 6 Population Proof

Table: **teaching_assistantship**
Rows loaded:
7

Sample rows (LIMIT 5)

```
AST-2025-0001   CS101   Introduction to Programming     1
AST-2025-0003   SE201   Software Architecture   0
AST-2025-0004   MA102   Linear Algebra  1
AST-2025-0007   PL101   Intro to Philosophy     1
AST-2025-0009   EC201   Macroeconomics  0
```

`research_assistantship`:

Step 6 Population Proof

Table: **research_assistantship**
Rows loaded:
5

Sample rows (LIMIT 5)

```
AST-2025-0002   Deep Learning on Edge   Model Quantization      1
AST-2025-0005   Quantum Optics Lab      Photon Entanglement     1
AST-2025-0006   Modern Ukrainian History Archiving       Digital Preservation    0
AST-2025-0008   Cognitive Overload Study        Attention Span Metrics  1
AST-2025-0012   Multilingual LLM Training       Zero-shot Translation   0
```

`workload_change_record`:

Step 6 Population Proof

Table: **workload_change_record**
Rows loaded:
3

Sample rows (LIMIT 5)

```
1       AST-2025-0001   2024-09-15 10:00:00     10      15      Prof. Oleksandr Tkachenko       Increased class size requires more grading time
2       AST-2025-0002   2024-10-01 14:30:00     20      10      Assoc. Prof. Andriy Shevchenko  Student requested workload reduction for midterms
3       AST-2025-0008   2025-02-10 09:15:00     20      15      Prof. Ihor Polischuk    Grant budget adjusted, reduced lab hours
```

7.3 Why Loaded Data Is Sufficient for Testing

Table	Sufficiency for testing
department	Multiple departments enable grouped report testing
supervisor	Multiple supervisors enable ownership and report joins
student	Enough rows for insert/delete and list operations
assistantship	Mixed types (TA/RA) and semesters enable filtering/aggregation

duty	Child records validate composite-key and delete chains
teaching_assistantship	TA specialization coverage
research_assistantship	RA specialization coverage
workload_change_record	Existing history enables audit report + update append validation

8. Step 7: Develop DBS Application: 3+ Forms for Data Manipulation

Inputs

- Step 1 input documents.
- Populated database from Step 6.

Outputs

- 3 integrated forms (add, update, delete).
- Query/queries per form.
- Before/after states of affected tables per form.

8.1 Form 1: Add Student

- UI endpoint: `/students/add`
- Input document: Student Profile Card
- SQL: in `sql/step7_forms_queries.sql`

Before/after evidence (from `sql/step9_test_execution.log`):

```
T1_BEFORE_COUNT = 0 for email step9.insert@ucu.edu.ua
T1_HTTP 302
T1_AFTER_ROW = 13  Step9  Insert  step9.insert@ucu.edu.ua
```

Screenshots:

- Before:

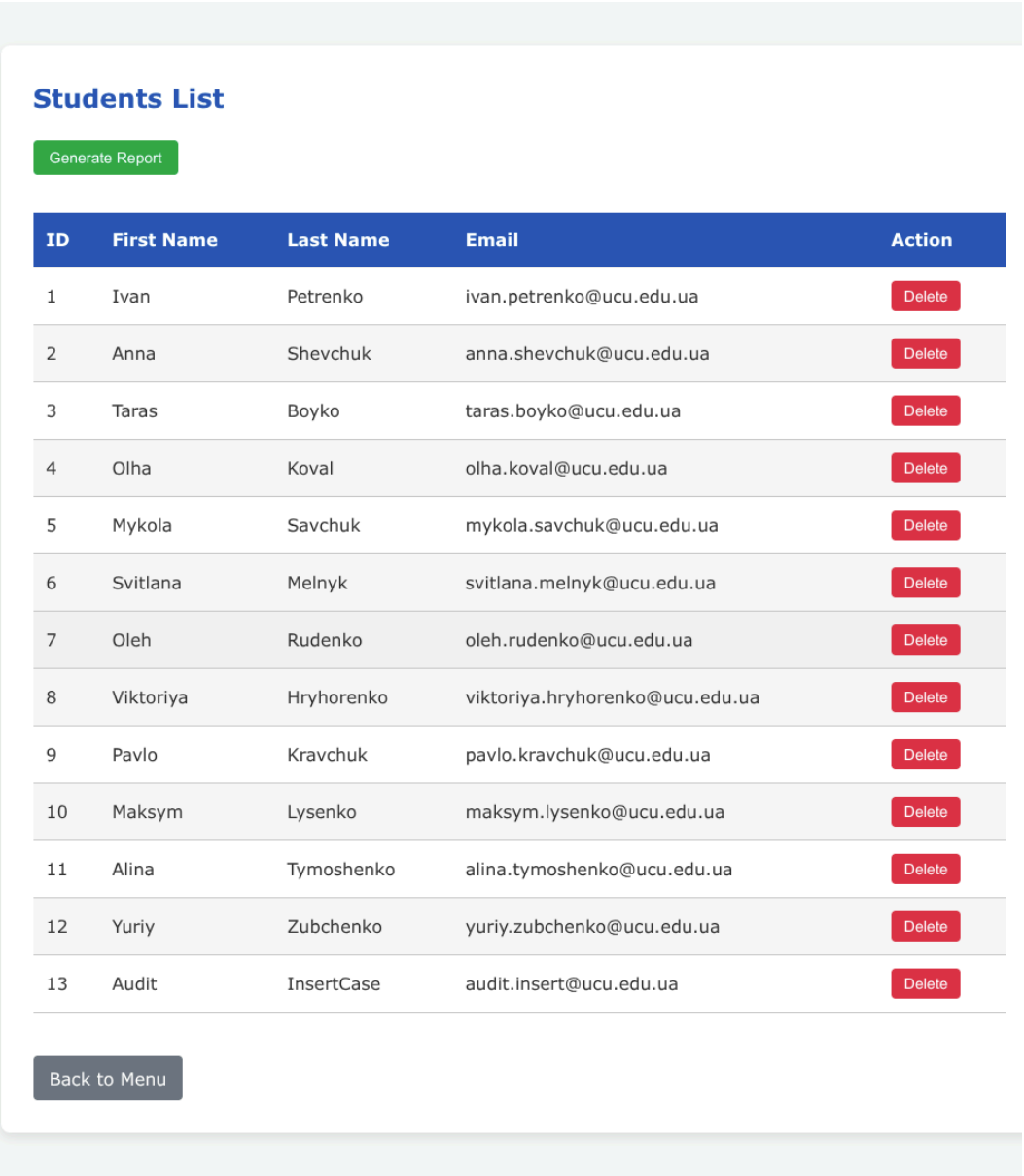
Students List

Generate Report

ID	First Name	Last Name	Email	Action
1	Ivan	Petrenko	ivan.petrenko@ucu.edu.ua	Delete
2	Anna	Shevchuk	anna.shevchuk@ucu.edu.ua	Delete
3	Taras	Boyko	taras.boyko@ucu.edu.ua	Delete
4	Olha	Koval	olha.koval@ucu.edu.ua	Delete
5	Mykola	Savchuk	mykola.savchuk@ucu.edu.ua	Delete
6	Svitlana	Melnyk	svitlana.melnyk@ucu.edu.ua	Delete
7	Oleh	Rudenko	oleh.rudenko@ucu.edu.ua	Delete
8	Viktoriya	Hryhorenko	viktoriya.hryhorenko@ucu.edu.ua	Delete
9	Pavlo	Kravchuk	pavlo.kravchuk@ucu.edu.ua	Delete
10	Maksym	Lysenko	maksym.lysenko@ucu.edu.ua	Delete
11	Alina	Tymoshenko	alina.tymoshenko@ucu.edu.ua	Delete
12	Yuriy	Zubchenko	yuriy.zubchenko@ucu.edu.ua	Delete

Back to Menu

- After:



Students List

Generate Report

ID	First Name	Last Name	Email	Action
1	Ivan	Petrenko	ivan.petrenko@ucu.edu.ua	Delete
2	Anna	Shevchuk	anna.shevchuk@ucu.edu.ua	Delete
3	Taras	Boyko	taras.boyko@ucu.edu.ua	Delete
4	Olha	Koval	olha.koval@ucu.edu.ua	Delete
5	Mykola	Savchuk	mykola.savchuk@ucu.edu.ua	Delete
6	Svitlana	Melnyk	svitlana.melnyk@ucu.edu.ua	Delete
7	Oleh	Rudenko	oleh.rudenko@ucu.edu.ua	Delete
8	Viktoriya	Hryhorenko	viktoriya.hryhorenko@ucu.edu.ua	Delete
9	Pavlo	Kravchuk	pavlo.kravchuk@ucu.edu.ua	Delete
10	Maksym	Lysenko	maksym.lysenko@ucu.edu.ua	Delete
11	Alina	Tymoshenko	alina.tymoshenko@ucu.edu.ua	Delete
12	Yuriy	Zubchenko	yuriy.zubchenko@ucu.edu.ua	Delete
13	Audit	InsertCase	audit.insert@ucu.edu.ua	Delete

Back to Menu

8.2 Form 2: Update Workload

- UI endpoint: `/assistantships/update`
- Input document: Assistantship Change Request
- SQL sequence: UPDATE assistantship + INSERT workload_change_record

Before/after evidence:

```
T2_BEFORE_ASSISTANTSHIP = AST-2025-0001  10
T2_HTTP 302
T2_AFTER_ASSISTANTSHIP = AST-2025-0001  18
T2_AFTER_LAST_LOG = ... old_hours=10 new_hours=18 reason='Step9 update verification'
```

Screenshots:

- Before:

Report: Workload Change Audit

Generate Report

Assistantship ID	Change Date	Old Hours	New Hours	Reason	Changed By
AST-2025-0008	2025-02-10T09:15	20	15	Grant budget adjusted, reduced lab hours	Prof. Ihor Polischuk
AST-2025-0002	2024-10-01T14:30	20	10	Student requested workload reduction for midterms	Assoc. Prof. Andriy Shevchenko
AST-2025-0001	2024-09-15T10:00	10	15	Increased class size requires more grading time	Prof. Oleksandr Tkachenko

Back to Menu

- After:

Report: Workload Change Audit

Generate Report

Assistantship ID	Change Date	Old Hours	New Hours	Reason	Changed By
AST-2025-0001	2026-04-13T13:34:25	10	18	Step7 screenshot update test	QA Supervisor
AST-2025-0008	2025-02-10T09:15	20	15	Grant budget adjusted, reduced lab hours	Prof. Ihor Polischuk
AST-2025-0002	2024-10-01T14:30	20	10	Student requested workload reduction for midterms	Assoc. Prof. Andriy Shevchenko
AST-2025-0001	2024-09-15T10:00	10	15	Increased class size requires more grading time	Prof. Oleksandr Tkachenko

Back to Menu

8.3 Form 3: Delete Student (with dependent data)

- UI endpoint: `/students/delete`
- Input document: Assistantship Termination Request
- SQL sequence: delete child rows first, then assistantship, then student

Before/after evidence:

```
T3_BEFORE_STUDENT = 1
T3_BEFORE_ASSISTANTSHIP = 1
T3_BEFORE_DUTY = 1
T3_BEFORE_TA = 1
T3_BEFORE_WCR = 1
T3_HTTP 302
T3_AFTER_STUDENT = 0
T3_AFTER_ASSISTANTSHIP = 0
T3_AFTER_DUTY = 0
T3_AFTER_TA = 0
T3_AFTER_WCR = 0
```

Screenshots:

- Before:

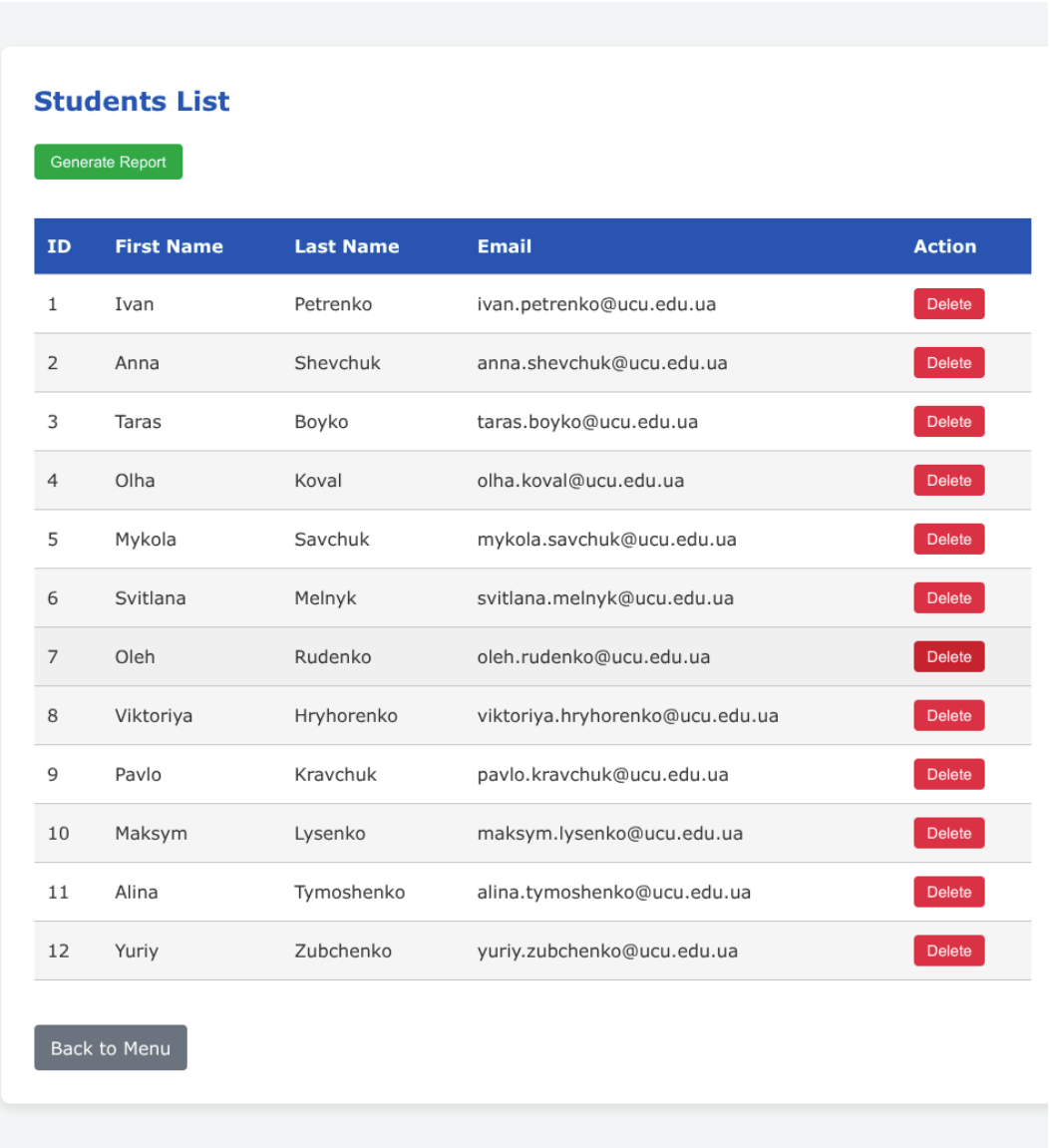
Students List

Generate Report

ID	First Name	Last Name	Email	Action
1	Ivan	Petrenko	ivan.petrenko@ucu.edu.ua	Delete
2	Anna	Shevchuk	anna.shevchuk@ucu.edu.ua	Delete
3	Taras	Boyko	taras.boyko@ucu.edu.ua	Delete
4	Olha	Koval	olha.koval@ucu.edu.ua	Delete
5	Mykola	Savchuk	mykola.savchuk@ucu.edu.ua	Delete
6	Svitlana	Melnyk	svitlana.melnyk@ucu.edu.ua	Delete
7	Oleh	Rudenko	oleh.rudenko@ucu.edu.ua	Delete
8	Viktoriya	Hryhorenko	viktoriya.hryhorenko@ucu.edu.ua	Delete
9	Pavlo	Kravchuk	pavlo.kravchuk@ucu.edu.ua	Delete
10	Maksym	Lysenko	maksym.lysenko@ucu.edu.ua	Delete
11	Alina	Tymoshenko	alina.tymoshenko@ucu.edu.ua	Delete
12	Yuriy	Zubchenko	yuriy.zubchenko@ucu.edu.ua	Delete
13	Audit	InsertCase	audit.insert@ucu.edu.ua	Delete

Back to Menu

- After:



Students List

Generate Report

ID	First Name	Last Name	Email	Action
1	Ivan	Petrenko	ivan.petrenko@ucu.edu.ua	Delete
2	Anna	Shevchuk	anna.shevchuk@ucu.edu.ua	Delete
3	Taras	Boyko	taras.boyko@ucu.edu.ua	Delete
4	Olha	Koval	olha.koval@ucu.edu.ua	Delete
5	Mykola	Savchuk	mykola.savchuk@ucu.edu.ua	Delete
6	Svitlana	Melnyk	svitlana.melnyk@ucu.edu.ua	Delete
7	Oleh	Rudenko	oleh.rudenko@ucu.edu.ua	Delete
8	Viktoriya	Hryhorenko	viktoriya.hryhorenko@ucu.edu.ua	Delete
9	Pavlo	Kravchuk	pavlo.kravchuk@ucu.edu.ua	Delete
10	Maksym	Lysenko	maksym.lysenko@ucu.edu.ua	Delete
11	Alina	Tymoshenko	alina.tymoshenko@ucu.edu.ua	Delete
12	Yuriy	Zubchenko	yuriy.zubchenko@ucu.edu.ua	Delete

Back to Menu

9. Step 8: Develop DBS Application: 3+ Reports for Output Documents

Inputs

- Output document requirements from Step 1.
- Populated database from Step 6.

Outputs

- 3 integrated reports.
- Application code with pipelined SQL queries.
- UI invocation screenshots and printout screenshots.

Report SQL is also collected in `sql/step8_report_queries.sql`.

9.1 Report 1: Assistantships by Department and Type

- UI endpoint: `/reports/by-department`
- Output document: Assistantship Allocation Summary
- Application query (from controller):

```
SELECT d.department_name, a.assistantship_type, COUNT(a.assistantship_id) AS total
FROM assistantship a JOIN department d ON a.department_id = d.department_id
GROUP BY d.department_name, a.assistantship_type
ORDER BY d.department_name;
```

Invocation screenshot:

Report: Assistantships by Department

Generate Report

Department Name	Assistantship Type	Count
Computer Science	TA	2
Data Science	RA	2
Economics	TA	1
History	RA	1
Mathematics	TA	1
Philosophy	TA	1
Physics	RA	1
Psychology	RA	1
Sociology	TA	1
Software Engineering	TA	1

Back to Menu

Printout screenshot:

Report: Assistantships by Department

Department Name	Assistantship Type	Count
Computer Science	TA	2
Data Science	RA	2
Economics	TA	1
History	RA	1
Mathematics	TA	1
Philosophy	TA	1
Physics	RA	1
Psychology	RA	1
Sociology	TA	1
Software Engineering	TA	1

9.2 Report 2: Workload Change Audit

- UI endpoint: `/reports/workload-changes`
- Output document: Workload Change Log
- Application query (from controller):

```
SELECT w.change_id, w.assistantship_id, a.student_id, w.change_date,
       w.old_hours, w.new_hours, w.reason, w.changed_by
FROM workload_change_record w JOIN assistantship a ON w.assistantship_id =
a.assistantship_id
ORDER BY w.change_date DESC;
```

Invocation screenshot:

Report: Workload Change Audit

Generate Report

Assistantship ID	Change Date	Old Hours	New Hours	Reason	Changed By
AST-2025-0001	2026-04-13T13:29:37	10	18	Step9_update_test	QA Supervisor
AST-2025-0008	2025-02-10T09:15	20	15	Grant budget adjusted, reduced lab hours	Prof. Ihor Polischuk
AST-2025-0002	2024-10-01T14:30	20	10	Student requested workload reduction for midterms	Assoc. Prof. Andriy Shevchenko
AST-2025-0001	2024-09-15T10:00	10	15	Increased class size requires more grading time	Prof. Oleksandr Tkachenko

Back to Menu

Printout screenshot:

Report: Workload Change Audit

Assistantship ID	Change Date	Old Hours	New Hours	Reason	Changed By
AST-2025-0001	2026-04-13T13:37:06	10	18	Step9 update verification	Step9 QA
AST-2025-0008	2025-02-10T09:15	20	15	Grant budget adjusted, reduced lab hours	Prof. Ihor Polischuk
AST-2025-0002	2024-10-01T14:30	20	10	Student requested workload reduction for midterms	Assoc. Prof. Andriy Shevchenko
AST-2025-0001	2024-09-15T10:00	10	15	Increased class size requires more grading time	Prof. Oleksandr Tkachenko

9.3 Report 3: Current Contracts per Student

- UI endpoint: `/reports/contracts`
- Output document: Current Contracts Summary
- Application query (from controller):

```
SELECT s.first_name, s.last_name, a.assistantship_id, a.hours_per_week, a.semester
FROM assistantship a JOIN student s ON a.student_id = s.student_id
ORDER BY s.last_name;
```

Invocation screenshot:

Report: Current Contracts

Generate Report

First Name	Last Name	Assistantship ID	Hours Per Week	Semester
Taras	Boyko	AST-2025-0003	15	Spring
Viktoriya	Hryhorenko	AST-2025-0008	20	Spring
Olha	Koval	AST-2025-0004	10	Spring
Pavlo	Kravchuk	AST-2025-0009	15	Summer
Maksym	Lysenko	AST-2025-0010	10	Fall
Svitlana	Melnyk	AST-2025-0006	15	Summer
Ivan	Petrenko	AST-2025-0001	18	Fall
Oleh	Rudenko	AST-2025-0007	10	Fall
Mykola	Savchuk	AST-2025-0005	20	Fall
Anna	Shevchuk	AST-2025-0002	20	Fall
Alina	Tymoshenko	AST-2025-0011	20	Spring
Yuriy	Zubchenko	AST-2025-0012	15	Fall

Back to Menu

Printout screenshot:

Report: Current Contracts

First Name	Last Name	Assistantship ID	Hours Per Week	Semester
Taras	Boyko	AST-2025-0003	15	Spring
Viktoriya	Hryhorenko	AST-2025-0008	20	Spring
Olha	Koval	AST-2025-0004	10	Spring
Pavlo	Kravchuk	AST-2025-0009	15	Summer
Maksym	Lysenko	AST-2025-0010	10	Fall
Svitlana	Melnyk	AST-2025-0006	15	Summer
Ivan	Petrenko	AST-2025-0001	18	Fall
Oleh	Rudenko	AST-2025-0007	10	Fall
Mykola	Savchuk	AST-2025-0005	20	Fall
Anna	Shevchuk	AST-2025-0002	20	Fall
Alina	Tymoshenko	AST-2025-0011	20	Spring
Yuriy	Zubchenko	AST-2025-0012	15	Fall

10. Step 9: Develop Tests and Test the DBS Application

Inputs

- In-scope tasks from Step 1.
- Implemented form/report queries from Steps 7 and 8.

Outputs

- Test plan.
- Executed tests with before/after proofs for data-change operations.
- Report correctness proofs using table data and query checks.

10.1 Test Plan

Test ID	Covered operation	Kind
T1	Register new data via add form	Data change
T2	Update workload via update form	Data change
T3	Delete student+dependent tuples via delete form	Data change
R1	Department/type allocation report	Report correctness

R2	Workload audit report	Report correctness
R3	Contracts report	Report correctness

10.2 Executed Data-Change Tests (Before/After)

Executable test runner: `sql/step9_run_tests.sh`
Execution log: `sql/step9_test_execution.log`

Key proof excerpts:

```
=== TEST 1: INSERT FORM ===
T1_BEFORE_COUNT -> 0
T1_HTTP -> 302
T1_AFTER_ROW -> inserted row exists

=== TEST 2: UPDATE FORM ===
T2_BEFORE_ASSISTANTSHIP -> AST-2025-0001 10
T2_HTTP -> 302
T2_AFTER_ASSISTANTSHIP -> AST-2025-0001 18
T2_AFTER_LAST_LOG -> appended change record

=== TEST 3: DELETE FORM ===
T3_BEFORE_* -> student/assistantship/child rows = 1
T3_HTTP -> 302
T3_AFTER_* -> student/assistantship/child rows = 0
```

Argument of success:

- Insert test succeeded because before count was 0 and target row appeared after POST.
- Update test succeeded because assistantship hours changed and workload history row was appended.
- Delete test succeeded because parent and all dependent rows were removed.

10.3 Report Correctness Proofs

Validation checks after tests:

```
assistantship_rows = 12
report_total_rows (sum of grouped allocation report) = 12
contracts_rows (join-based contracts report) = 12
```

Queries used for proof (see `sql/step9_tests.sql`):

```sql
SELECT COUNT(*) AS assistantship_rows FROM assistantship;

SELECT SUM(x.total) AS report_total_rows
FROM (
  SELECT COUNT(*) AS total
  FROM assistantship a
  JOIN department d ON a.department_id = d.department_id
  GROUP BY d.department_name, a.assistantship_type
```

```sql
) x;

SELECT COUNT(*) AS contracts_rows
FROM (
  SELECT a.assistantship_id
  FROM assistantship a
  JOIN student s ON a.student_id = s.student_id
) q;
```

Additional workload-audit data proof query (same file):

```sql
SELECT change_id, assistantship_id, old_hours, new_hours, reason
FROM workload_change_record
WHERE assistantship_id = 'AST-2025-0001'
ORDER BY change_id DESC
LIMIT 1;
```

Argument of correctness:

- R1 aggregation is correct because grouped totals sum to the base assistantship cardinality.
- R2 is correct because latest workload audit row reflects the tested update (10 -> 18).
- R3 is correct because one contract row is generated per assistantship in the joined data.

11. Summary

The Assistantships DBS was completed through all HW3 steps with traceable artifacts and executed evidence:

- stakeholder-scoped modeling,
- schema refinement and 3NF proofs,
- populated data sufficient for functional tests,
- 3 required forms and 3 required reports integrated into the application,
- reproducible Step 9 tests with before/after proofs for data-changing operations and report correctness checks.

12. Conclusion

The implementation and report were updated and revalidated against the latest Lab 9/Lab 10 descriptions, HW3 guidelines, and grading penalties. In addition to functional verification, per-table visual proofs for Step 4 and Step 6 were added to reduce grading risk where penalties are applied per missing table-level evidence.

13. References

1. DB-HW3_Guidelines.pdf
2. HW3 Grading Penalties by Steps.pdf
3. Lab 9 materials (scoping, external views, harmonization)
4. Lab 10 materials (schema refinement, normalization, population)
5. Repository implementation artifacts in sql and src/main
6. Repository URL: https://github.com/dark-neonus/db_hw_3_pasichnyk

14. Annex

A. SQL Artifacts

- `sql/step4_refined_schema.sql`
- `sql/step6_populate_data.sql`
- `sql/step7_forms_queries.sql`
- `sql/step8_report_queries.sql`
- `sql/step9_tests.sql`
- `sql/step9_run_tests.sh`
- `sql/step9_test_execution.log`

B. Application Artifacts

- `src/main/java/ua/edu/ucu/db/hw3/assistantships/WebController.java`
- `src/main/resources/templates/index.html`
- `src/main/resources/templates/students_add.html`
- `src/main/resources/templates/students_list.html`
- `src/main/resources/templates/workload_update.html`
- `src/main/resources/templates/report_by_department.html`
- `src/main/resources/templates/report_workload_changes.html`
- `src/main/resources/templates/report_contracts.html`

C. Evidence Screenshots

- Step 4: `screenshots/step4/`
- Step 6: `screenshots/step6/`
- Step 7: `screenshots/step7/`
- Step 8: `screenshots/step8/`

D. Raw Execution Evidence

- `evidence/step4/`
- `evidence/step6/`